

Team Challenge: Sprint 03-04 - Películas (E3)

1. Introducción

El objetivo de este documento es describir la organización del proyecto de **tc-sprint03-04-peliculas-E3** correspondiente al primer *Team Challenge* del bootcamp de **AI Engineering** de **The Bridge**, así como información técnica relevante para que los integrantes del equipo la puedan tener a mano.

2. Equipo de Programadores

El equipo está compuesto por los siguientes programadores, todos con el mismo nivel de responsabilidad:

- **Lucas Ávila Nebreda**
- **Adixon Alexix Díaz Sánchez**
- **Alejandra del Carme Eng Broca**
- **Raúl Molina Ordoñez**

Scrum Master

El Scrum Master, y por tanto el responsable de crear el repositorio original y validar las *Pull Requests* del resto de compañeros es:

- **Lucas Ávila Nebreda**
-

3. Reunión de Kick off

El **19 de junio de 2026** se realizó la primera reunión de *kick off* del proyecto.

En dicha reunión se acordó que, dado que este era un proyecto académico y el objetivo era absorber el máximo de conocimientos posible, **todos los programadores iban a realizar todos los ejercicios de la práctica**, y antes de la entrega se decidiría qué versión o versiones eran las más adecuadas para presentar como entrega definitiva.

Para no desvirtuar el uso de Git y su aprendizaje, se decidió usar la siguiente estrategia:

- Tal como marca la tarea, se tendría rama **main** y rama **develop**.
 - Tal como marca la tarea, se trabajaría por ramas, pero las ramas se distribuirían por **nombre de programador**.
 - Todos los programadores programarían sobre un único fichero:
Team_Challenge_sprint03_04.ipynb.
 - Se crearía, dentro de **Team_Challenge_sprint03_04.ipynb**, un **"code cell" (celda de código)** para **cada uno de los programadores y cada ejercicio**. De esta forma, aun tocando el mismo fichero, no se encontrarán conflictos graves que resolver en los *Pull Requests* hacia **develop**.
 - **Flujo de trabajo con Git:** Cada vez que un programador avanzara en una tarea, realizaría un *pull* de **develop** en local, mergearía **develop** con su rama, haría un *push* de su rama a GitHub y abriría un *Pull Request* a **develop**.
-

4. Repositorio y Ramas

Enlace al repositorio oficial: <https://github.com/LucasAvilal/tc-sprint03-04-peliculas-E3>

Estructura de carpetas inicial

```
|— data/ |— .env.prueba |— .gitignore
|— requirements.txt
|— Team_Challenge_sprint03_04.ipynb
```

5. Ramas: Estructura y su responsable

Nombre de la rama	Programador responsable
Adixon	Adixon Alexix Díaz Sánchez
Alejandra	Alejandra del Carme Eng Broca
LucasAvilal-patch-1	Lucas Ávila Nebreda1
Raul	Raúl Molina Ordoñez1

6. Arquitectura y Fases del Proyecto Completo

El desarrollo se divide en 5 grandes bloques interconectados dentro del cuaderno individual:

A. Ingesta y Extracción de Datos (TMDB API)

- Configuración de la conexión con la API de *The Movie Database* (TMDB).
- Construcción de peticiones paginadas y manejo de *rate-limiting* (límites de velocidad).
- Extracción y estructuración inicial de metadatos de películas en formato JSON y posterior conversión a DataFrames.

B. Análisis Exploratorio de Datos (EDA) y Limpieza

- Identificación y tratamiento de valores nulos, registros duplicados y tipos de datos inconsistentes.
- Análisis estadístico descriptivo de variables numéricas y categóricas (presupuestos, ingresos, géneros, puntuaciones).
- Visualización de distribuciones y correlaciones iniciales para entender el comportamiento de los datos.

C. Ingeniería de Variables y Transformación

- Preprocesamiento de texto y normalización de variables categóricas (ej. extracción de géneros y palabras clave).
- Escalado de variables numéricas y codificación (*encoding*) para preparar los datos de entrada al modelo.
- Creación de nuevas características estructuradas que maximicen el rendimiento del algoritmo.

D. Modelado y Entrenamiento de IA

- División del conjunto de datos en entrenamiento, validación y prueba (*train/test split*).

- Selección, configuración y entrenamiento del algoritmo principal del Sprint.
- Optimización de hiperparámetros para mejorar la capacidad de generalización del modelo.

E. Evaluación y Conclusiones del Negocio

- Evaluación del rendimiento utilizando métricas clave del sector (ajustadas al tipo de problema: regresión o clasificación).
- Análisis de errores para identificar sesgos o debilidades del modelo entrenado.
- Conclusiones técnicas e implicaciones prácticas de la solución de IA sobre el catálogo de películas.

7. Tecnologías y Herramientas Utilizadas

- **Lenguaje:** Python 3.x
- **Entorno:** Jupyter Notebooks ([.ipynb](#))
- **Librerías Clave:** Pandas, NumPy, Requests (API), Scikit-Learn, Matplotlib / Seaborn.
- **Control de Versiones:** Git & GitHub

8. Clonar repositorio y activar entorno virtual

En terminal donde queramos guardar el repo del proyecto:

```
git clone https://github.com/vuestro-usuario/tc-sprint03-04.git
cd tc-sprint03-04
python -m venv venv
source venv/bin/activate      # Mac/Linux
venv\Scripts\activate        # Windows
pip install -r requirements.txt
cp .env.example .env
```

9. Workflow push rama

```
git add .
git commit -m "feat: Lo que sea"
git switch develop
git pull origin develop
git switch rama_del_programador
git merge develop
git push origin rama_del_programador
```

10. Sintaxis commits

Prefijo	Como usarlo	ejemplo
feat:	Nueva funcionalidad	feat: función fetch_movie_details con TMDB
fix:	Corrección de bug	fix: manejar tmdbId nulo en links.csv
docs:	Documentación	docs: añadir instrucciones de claves en README
refactor:	Refactorización sin cambio funcional	refactor: extraer groupby a función separada
chore:	Tareas de mantenimiento	chore: actualizar requirements.txt